C Assignment Debugging & Analysis Report

📂 1. File: 1.c — Student Marks & General Array Validations

🎯 PROGRAM INTENT

The program manages a student statistics profile. It computes total and average marks, maps internal scores to specific lookup indices, mirrors array variables to temporary buffers, and tracks execution parameters.

🔍 CHECKLIST BREAKDOWN & VULNERABILITIES FOUND

- **Syntax Errors:** The variable declaration statement `int marks[5] = {80,90,70,60,50}` is missing its terminating semicolon. Additionally, the statements `attendance["one"] = 1;` and attendance[2.5] = 1; leverage string constants and floating-point parameters as subscripts, which is explicitly prohibited for standard array indices.
- **Semantic Errors & Type Mismatches:** The string array assignment grade[0] = "A"; attempts to assign a multi-byte string literal address pointer to a singular character target byte. Direct pointer variable assignments such as inventory = prices; and city = "Jaipur"; violate the unmodifiable address property of array bases.
- **Runtime & Array Bounds Errors:** The loops bounded by for(int i=0; i<=5; i++) result in off-by-one iterations that attempt to fetch indices past the defined size limits of the array structures, prompting unauthorized memory reading vulnerabilities. The initialization scores[5] = 60; breaks bounds rules for a size-5 array.
- **Buffer Overflows:** Processing strcpy(subject,"Programming"); inside a 5-byte target buffer truncates critical boundaries and overwrites stack memory.
- **Logical Errors:** The statement average = total / 5; leads to implicit integer division truncation prior to assignment. The standard data binding call scanf("%d", arr[i]); passes a raw element value instead of providing the target element address. Equality conditions are written using an assignment operator `=` instead of ==. Mismatched format specifiers use integer tokens %d to print floating-point types. The `grade` character array is missing an explicit string null-terminator.

💻 CORRECTED SOURCE CODE

```
#include <stdio.h>
#include <string.h>
```

```
int main()
{
    int marks[5] = {80, 90, 70, 60, 50};

    printf("Student Marks System\n");

    int total = 0;
    for(int i = 0; i < 5; i++)
    {
        total += marks[i];
    }

    printf("Total = %d\n", total);

    float average;
    average = total / 5.0;

    printf("Average = %.2f\n", average);
    printf("\n");

    int    scores[6];
    scores[0]  =  10;
    scores[1]  =  20;
    scores[2]  =  30;
    scores[3]  =  40;
    scores[4]  =  50;
    scores[5] = 60;

    printf("Scores\n");
    for(int i = 0; i < 6; i++)
    {
        printf("%d ", scores[i]);
    }
    printf("\n\n");

    char subject[12];
    strcpy(subject, "Programming");
    printf("%s\n", subject);
    printf("\n");

    int attendance[5];
    for(int i = 0; i < 5; i++)
    {
        attendance[i] = i;
    }

    printf("Attendance\n");
    for(int i = 0; i < 5; i++)
    {
        printf("%d ", attendance[i]);
```

```
}
printf("\n\n");

int arr[5];
printf("Enter 5integers:\n");
for(int i = 0; i < 5; i++)
{
    scanf("%d",&arr[i]);
}
printf("\n");

int totalMarks = 0;
for(int i = 0; i < 5; i++)
{
    totalMarks += arr[i];
}
printf("Total   Marks=  %d\n", totalMarks);

char    grade[3];
grade[0]   =   'A';
grade[1]   =   '+';
grade[2] = '\0';

printf("%s\n", grade);
printf("\n");

intprices[5]    ={100,    200, 300, 400, 500};

if(prices[2] == 300)
{
    printf("Price Found\n");
}
printf("\n");

int inventory[5];
for(int i = 0; i < 5; i++)
{
    inventory[i] =prices[i];
}
printf("%d\n", inventory[0]);
printf("\n");

float salary[5];
salary[0] = 50000.50;
printf("%.2f\n", salary[0]);
printf("\n");

char city[10];
strcpy(city, "Jaipur");
printf("%s\n", city);
```

```
    printf("\n");

    int result[5];
    for(int i = 0; i < 5; i++)
    {
        result[i] = i * 10;
    }

    printf("Results\n");
    for(int i = 0; i < 5; i++)
    {
        printf("%d ", result[i]);
    }
    printf("\n\n");

    printf("Program Finished\n");
    return 0;
}
```

📂 2. File: 2.c — Mathematical Aggregations & Highest Bounds Evaluation

🎯 PROGRAM INTENT

The program populates index sequences, tracks accumulated sums across numerical inputs, isolates peak maximum criteria points within data series, and formats floating-point values.

🔍 CHECKLIST BREAKDOWN & VULNERABILITIES FOUND

- **Syntax & Compiler Errors:** The ultimate output printing statement lacks a statement termination symbol. Extraneous textual indices definitions (e.g., fees["two"]) fail base types verification.
- **Runtime & Array Bounds Errors:** Array lookup sequences structured as for(int i=0; i<=10; i++) extend directly over boundaries, pulling unallocated heap garbage and generating random stability outcomes.
- **Logical & Structural Flaws:** The math operations block replaces accumulated variables via an incorrect assignment statement sum = numbers[i]; instead of updating using addition logic. The peak discovery block uses an inverted validation operator <, causing it to inadvertently capture minimum results instead of maximum points. Text conversions and raw float outputs mismatch placeholders by parsing parameters through integer flags. String sequence blocks lack explicit null characters.

💻 CORRECTED SOURCE CODE

```
#include <stdio.h>
#include <string.h>

int main()
{
    int numbers[10];
    printf("Array Operations Program\n");

    for(int i = 0; i < 10; i++)
    {
        numbers[i] = (i + 1) * 10;
    }

    printf("\nNumbers Array\n");
    for(int i = 0; i < 10; i++)
    {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    int sum = 0;
    for(int i = 0; i < 10; i++)
    {
```

```
    sum += numbers[i];
}
printf("Sum= %d\n",sum);

float average;
average = sum / 10.0;
printf("Average=   %.2f\n", average);
printf("\n");

intmarks[5]=    {90,85,    80, 75, 70};
printf("Highest Marks\n");

int highest = marks[0];
for(int i = 1; i < 5; i++)
{
    if(marks[i] > highest)
    {
        highest= marks[i];
    }
}
printf("%d\n", highest);
printf("\n");

intattendance[6]  ={1,    1, 1, 0, 1, 1};
int present = 0;
for(int i = 0; i < 6; i++)
{
    present+= attendance[i];
}
printf("PresentDays=    %d\n", present);
printf("\n");

char subject[16];
strcpy(subject,"ComputerScience");
printf("%s\n", subject);
printf("\n");

char branch[20];
strcpy(branch, "IT");
printf("%s\n", branch);
printf("\n");

char     college[6];
college[0]   =   'A';
college[1]   =   'C';
college[2]   =   'E';
college[3]    =    'I';
college[4]   =   'T';
college[5] = '\0';
```

```
    printf("%s\n", college);
    printf("\n");

    int fees[5];
    for(int i = 0; i < 5; i++)
    {
        fees[i] = i * 10000;
    }

    printf("FeesStructure\n");
    for(int i = 0; i < 5; i++)
    {
        printf("%d  ",   fees[i]);
    }
    printf("\n\n");

    int result[5];
    printf("Enter   5result integers:\n");
    for(int i = 0; i < 5; i++)
    {
        scanf("%d",&result[i]);
    }
    printf("\n");

    int totalResult = 0;
    for(int i = 0; i < 5; i++)
    {
        totalResult+=result[i];
    }
    printf("%d\n", totalResult);
    printf("\n");

    intstock[5]=     {100,200, 300, 400, 500};
    if(stock[2] == 300)
    {
        printf("Stock   Available\n");
    }
    printf("\n");

    int backup[5];
    for(int i = 0; i < 5; i++)
    {
        backup[i] =stock[i];
    }
    printf("%d\n", backup[0]);
    printf("\n");

    float salary[5];
    salary[0] = 25000.75;
    printf("%.2f\n", salary[0]);
```

```
    printf("\n");

    int inventory[5];
    for(inti    =0;   i  <5; i++)
    {
        inventory[i] = i;
    }

    printf("Inventory\n");
    for(inti    =0;   i  <5; i++)
    {
        printf("%d ", inventory[i]);
    }
    printf("\n\n");

    char city[11];
    strcpy(city,"JaipurCity");
    printf("%s\n", city);
    printf("\n");

    int count[5];
    for(inti    =0;   i  <5; i++)
    {
        count[i]= inventory[i];
    }
    printf("%d\n", count[0]);
    printf("\n");

    printf("Program Ended\n");
    return 0;
}
```

📂 3. File: 3.c — Statistical Extrema & IO Stream Binding

🎯 PROGRAM INTENT

The program manages educational tracking schemas, recording minimum metrics, reading branch parameters from standard interface nodes, and safeguarding array variables against index out-of-bounds occurrences.

🔍 CHECKLIST BREAKDOWN & VULNERABILITIES FOUND

- **Runtime Bounds Faults:** Output monitoring loops and assignment iteration tracking run over valid data boundaries. Memory sizes for data blocks handling string literals are under-allocated, yielding access violations.
- **Logical Mismatches:** The verification logic tracking minimum marks uses an incorrect comparison operator >, which calculates the wrong metric.
- **Semantic Pointer Errors:** The argument structure scanf("%s", &branch); mistakenly targets a multi-tier absolute reference address pointer rather than passing the simple identifier of the string block.

💻 CORRECTED SOURCE CODE

```
#include <stdio.h>
#include <string.h>

int main()
{
    intmarks[5]=    {85,90,       75, 60, 95};
       printf("StudentResult Analysis\n");

    int total = 0;
    for(int i = 0; i < 5; i++)
    {
        total += marks[i];
    }
    printf("Total =%d\n", total);

    float average = total / 5.0;
    printf("Average= %f\n", average);
    printf("\n");

    int highest = marks[0];
    for(int i = 1; i < 5; i++)
    {
        if(marks[i] > highest)
        {
            highest= marks[i];
        }
    }
```

```
    printf("Highest=   %d\n", highest);
    printf("\n");

    int lowest = marks[0];
    for(int i = 1; i < 5; i++)
    {
        if(marks[i]<    lowest)
        {
            lowest = marks[i];
        }
    }
    printf("Lowest = %d\n", lowest);
    printf("\n");

    intattendance[5]  ={1,    1,   0,  1,   1};
    int present = 0;
    for(int i = 0; i < 5; i++)
    {
        present+=  attendance[i];
    }
    printf("Present=   %d\n", present);
    printf("\n");

    char subject[12];
    strcpy(subject,"Programming");
    printf("%s\n", subject);
    printf("\n");

    char branch[10];
    printf("Enter  BranchName:\n");
    scanf("%s", branch);
    printf("%s\n", branch);
    printf("\n");

    int          fees[6];
    fees[0]  =  50000;
    fees[1]  =  60000;
    fees[2]  =  70000;
    fees[3]  =  80000;
    fees[4]  =  90000;
    fees[5] = 100000;

    printf("Fees\n");
    for(int i = 0; i < 6; i++)
    {
        printf("%d  ",  fees[i]);
    }
    printf("\n\n");

    intstock[5]=     {10,20,    30,  40,  50};
```

```
if(stock[2] == 30)
{
    printf("Stock Available\n");
}
printf("\n");

int backup[5];
for(int i = 0; i < 5; i++)
{
    backup[i] =stock[i];
}
printf("%d\n", backup[0]);
printf("\n");

float salary[5];
salary[0] = 45000.75;
printf("%.2f\n", salary[0]);
printf("\n");

int result[5];
printf("Enter 5result points:\n");
for(int i = 0; i < 5; i++)
{
    scanf("%d",&result[i]);
}
printf("\n");

int totalResult = 0;
for(int i = 0; i < 5; i++)
{
    totalResult+=result[i];
}
printf("Total   Result=   %d\n", totalResult);
printf("\n");

char city[10];
strcpy(city, "Jaipur");
printf("%s\n", city);
printf("\n");

int inventory[5];
for(int i = 0; i < 5; i++)
{
    inventory[i] =i* 100;
}

printf("Inventory\n");
for(int i = 0; i < 5; i++)
{
    printf("%d ", inventory[i]);
```

```
    }
    printf("\n\n");

    char            college[6];
    college[0]       =       'A';
    college[1]       =       'C';
    college[2]       =       'E';
    college[3]       =        'I';
    college[4]       =       'T';
    college[5]       =       '\0';
    printf("%s\n",    college);
    printf("\n");

    int count[5];
    for(int i = 0; i < 5; i++)
    {
        count[i]= inventory[i];
    }
    printf("%d\n", count[0]);
    printf("\n");

    char course[9];
    strcpy(course, "Computer");
    printf("%s\n", course);
    printf("\n");

    int numbers[5];
    for(int i = 0; i < 5; i++)
    {
        numbers[i] = i * 10;
    }

    printf("Numbers\n");
    for(int i = 0; i < 5; i++)
    {
        printf("%d ", numbers[i]);
    }
    printf("\n\n");

    printf("ProgramFinished\n");
    return 0;
}
```

📂 4. File: 4.c — Financial Transaction Reports & Boundary Analysis

🎯 PROGRAM INTENT

The system builds transaction validation summaries, evaluates maximum sales targets, runs index mapping loops across employee matrices, and safely processes corporate text identifiers.

🔍 CHECKLIST BREAKDOWN & VULNERABILITIES FOUND

- **Runtime Array Overruns:** The expression `ratings[-1] = 5;` targets illegal memory addresses outside array definitions. The assignment `monthlySales[5] = 250;` overflows an array of size 5.
- **Logical Evaluator Failures:** The comparison check tracking maximum performance statistics uses an inverted operator `<`, recording minimal transaction data instead.
- **Print Formatting Specifier Errors:** Passing a primitive integer parameter into `printf("%s ", feedback[0]);` using a string format specifier leads to runtime memory violation crashes.

💻 CORRECTED SOURCE CODE

```
#include <stdio.h>
#include <string.h>

int main()
{
    intsales[5]=    {100,200,   300, 400, 500};
    printf("Sales ReportSystem\n");

    int totalSales = 0;
    for(int i = 0; i < 5; i++)
    {
        totalSales += sales[i];
    }
    printf("Total   Sales=   %d\n", totalSales);

    float  averageSales=totalSales / 5.0;
    printf("AverageSales   = %.2f\n", averageSales);
    printf("\n");

    inthighestSale= sales[0];
    for(int i = 1; i < 5; i++)
    {
        if(sales[i]> highestSale)
        {
            highestSale= sales[i];
        }
    }
```

```
printf("Highest Sale = %d\n", highestSale);
printf("\n");

intmonthlySales[6]  =  {120,  140,  160,  180,  200,  250};
printf("Monthly Sales\n");
for(int i = 0; i < 6; i++)
{
    printf("%d ", monthlySales[i]);
}
printf("\n\n");

char product[15];
strcpy(product, "LaptopComputer");
printf("%s\n", product);
printf("\n");

char category[15];
strcpy(category, "Electronics");
printf("%s\n", category);
printf("\n");

int stock[5];
for(int i = 0; i < 5; i++)
{
    stock[i] = i * 50;
}

printf("Stock    Details\n");
for(int i = 0; i < 5; i++) {

    printf("%d ", stock[i]);
}
printf("\n\n");

int orders[5];
printf("Enter 5 order details values:\n");
for(int i = 0; i < 5; i++)
{
    scanf("%d", &orders[i]);
}
printf("\n");

inttotalOrders = 0;
for(int i = 0; i < 5; i++)
{
    totalOrders += orders[i];
}
printf("Total Orders = %d\n", totalOrders);
printf("\n");
```

```
intrevenue[5]   = {1000,  2000, 3000, 4000, 5000};
if(revenue[2] == 3000)
{
     printf("RevenueMatched\n");
}
printf("\n");

int backupRevenue[5];
for(int i = 0; i < 5; i++)
{
     backupRevenue[i]= revenue[i];
}
printf("%d\n", backupRevenue[0]);
printf("\n");

float commission[5];
commission[0] = 2500.50;
printf("%.2f\n", commission[0]);
printf("\n");

char city[10];
strcpy(city, "Jaipur");
printf("%s\n", city);
printf("\n");

char branch[5] = {'I',     'T', '-', 'A', '\0'};
printf("%s\n", branch);
printf("\n");

int employees[5];
for(int i = 0; i < 5; i++)
{
     employees[i] =i* 100;
}

printf("Employees\n");
for(int i = 0; i < 5; i++)
{
     printf("%d ", employees[i]);
}
printf("\n\n");

char department[10];
strcpy(department,"Marketing");
printf("%s\n", department);
printf("\n");

int expenses[5];
for(int i = 0; i < 5; i++)
{
```

```
        expenses[i] = sales[i];
    }
    printf("%d\n", expenses[0]);
    printf("\n");

    int ratings[5];
    ratings[0]          =          5;
    printf("%d\n",     ratings[0]);
    printf("\n");

    intfeedback[5] = {10, 20, 30, 40, 50};
    printf("%d\n", feedback[0]);
    printf("\n");

    printf("Report Generated Successfully\n");
    return 0;
}
```

📂 5. File: 5.c — Institutional Audits & Data Pipeline Controls

🎯 PROGRAM INTENT

The program runs analysis checks on large grading indices, tracks standard performance milestones, handles temporary storage loops, and enforces secure string parameters copy validations.

🔍 CHECKLIST BREAKDOWN & VULNERABILITIES FOUND

- **Logical Extrema Flaws:** The evaluation structures seeking high and low performance scores are inverted. The maximum tracking operation uses <, while the minimum verification loop leverages >, rendering both outputs invalid.
- **Runtime Sizing Overflows:** The storage size allocated for char department[10]; cannot fit the string "InformationTechnology", generating buffer overruns. Iterations across loops managing data assignments contain off-by-one errors.

💻 CORRECTED SOURCE CODE

```
#include <stdio.h>
#include <string.h>

int main()
{
    int marks[10] = {75, 80, 85, 90, 95, 70, 65, 88, 92, 78};
    printf("Student Performance Analysis\n");

    int total = 0;
    for(int i = 0; i < 10; i++)
    {
        total += marks[i];
    }
    printf("Total Marks = %d\n", total);

    float average = total / 10.0;
    printf("Average = %.2f\n", average);
    printf("\n");

    int highest = marks[0];
    for(int i = 1; i < 10; i++)
    {
        if(marks[i] > highest)
        {
            highest = marks[i];
        }
    }
    printf("Highest = %d\n", highest);
```

```
printf("\n");

int lowest = marks[0];
for(inti =1; i <10; i++)
{
     if(marks[i]< lowest)
     {
          lowest = marks[i];
     }
}
printf("Lowest = %d\n", lowest);
printf("\n");

int attendance[11];
for(inti =0; i <10; i++)
{
     attendance[i] = 1;
}
attendance[10] = 0;

int present = 0;
for(inti =0; i <11; i++)
{
     present+= attendance[i];
}
printf("PresentDays=    %d\n", present);
printf("\n");

char subject[15];
strcpy(subject,"DataStructures");
printf("%s\n", subject);
printf("\n");

char department[25];
strcpy(department,"InformationTechnology");
printf("%s\n", department);
printf("\n");

int fees[5];
for(int i = 0; i < 5; i++)
{
     fees[i] = i * 10000;
}

printf("Fees\n");
for(int i = 0; i < 5; i++)
{
     printf("%d ", fees[i]);
}
printf("\n\n");
```

```
int result[5];
printf("Enter  5result parameters:\n");
for(inti    =0;   i  <5; i++)
{
    scanf("%d",&result[i]);
}
printf("\n");

int totalResult = 0;
for(inti    =0;   i  <5; i++)
{
    totalResult+=result[i];
}
printf("%d\n", totalResult);
printf("\n");

intinventory[5]   ={10, 20, 30, 40, 50};
if(inventory[2] == 30)
{
    printf("Item Found\n");
}
printf("\n");

int backup[5];
for(inti    =0;   i  <5; i++)
{
    backup[i] =inventory[i];
}
printf("%d\n", backup[0]);
printf("\n");

float salary[5];
salary[0] = 65000.75;
printf("%.2f\n", salary[0]);
printf("\n");

char city[10];
strcpy(city, "Jaipur");
printf("%s\n", city);
printf("\n");

char branch[5] =  {'I', 'T', '-', 'A', '\0'};
printf("%s\n", branch);
printf("\n");

int stock[5];
for(inti    =0;   i  <5; i++)
{
    stock[i] = i * 100;
```

```
    }

    printf("Stock\n");
    for(inti    =0;   i  <5; i++)
    {
        printf("%d ", stock[i]);
    }
    printf("\n\n");

    char course[12];
    strcpy(course, "Programming");
    printf("%s\n", course);
    printf("\n");

    int ratings[5];
    ratings[0] = 10;
    printf("%d\n", ratings[0]);
    printf("\n");

    intfeedback[5]=    {1, 2, 3, 4, 5};
    printf("%d\n", feedback[0]);
    printf("\n");

    int scores[5];
    for(inti    =0;   i  <5; i++)
    {
        scores[i] =marks[i];
    }
    printf("%d\n", scores[0]);
    printf("\n");

    int temp[6];
    for(inti    =0;   i  <6; i++)
    {
        temp[i] = i;
    }
    printf("%d\n", temp[5]);
    printf("\n");

    printf("ProgramCompleted Successfully\n");
    return 0;
}
```